١ / الصورة الكبيرة

الفكرة الأساسية: Application يصف التقرير، وInfrastructure ترسمه. التطبيق لا يستورد QuestPDF ولا يعرف تفاصيلها.

١ **HTTP Client**
يطلب ملف PDF من endpoint.

↓

٢ **WebApi**
يستقبل الطلب، يختار خدمة المثال، ثم يرجع PDF bytes كملف.

↓

٣ **Application**
يبني تعريفًا محايدًا: صفوف، أعمدة، ترتيب، ومجاميع.

↓

٤ **IPdfReportRenderer**
عقد يفصل تعريف التقرير عن تقنية الرسم.

↓

٥ **Infrastructure / QuestPdfReportRenderer**
يحوّل التعريف إلى صفحة وجدول وينتج الملف.

القاعدة المختصرة

```
Application → عقد محايد ← Infrastructure adapter
```

٢ / حدود الطبقات

WebApi | واجهة HTTP

مسؤوليته: routes وحقن الخدمات وإرجاع الملف. لا يبني layout ولا يستدعي QuestPDF.

ملفات

```
ReportPoC.WebApi/Program.cs
ReportPoC.WebApi/Endpoints/ReportEndpoints.cs
```

Application | تعريف التقرير

مسؤوليته: بيانات المثال، Fluent Builder، تعريف flat/grouped، وعقد IPdfReportRenderer. لا يعرف مكتبة PDF.

ملفات

```
ReportPoC.Application/Reporting/PdfReport.cs
ReportPoC.Application/Reporting/GroupedPdfReport.cs
ReportPoC.Application/Reporting/IPdfReportRenderer.cs
```

Infrastructure | الرسم الفعلي

مسؤوليته: تنفيذ العقد باستخدام QuestPDF، تحويل العرض والألوان والخلايا، وإنتاج byte[].

ملفات

```
ReportPoC.Infrastructure/Reporting/QuestPdfReportRenderer.cs
```

Tests | التحقق من السلوك

تختبر Builder وخدمات Application باستخدام fake renderer؛ وتتحقق smoke test من PDF الفعلي.

ملفات

```
ReportPoC.Application.Tests/Reporting/*
```

اتجاه الاعتماد: WebApi يعرف Application وInfrastructure للتسجيل. Infrastructure تعتمد على Application لتنفيذ العقد. Application لا تعتمد على QuestPDF.

٣ / رحلة الطلب: إطارات متتابعة

كل بطاقة إطار من الحركة: اقرأها بالتتابع لتتبع طلب GET حتى bytes الملف.

FRAME 1

GET /reports/grouped

المستخدم يطلب التقرير المجمع.

↓

FRAME 2

ReportEndpoints

ينادي GenerateGroupedAsync ويمرر CancellationToken.

↓

FRAME 3

SampleSalesReportService

ينشئ مجموعات العينة ويختار الأعمدة وإعداداتها.

↓

FRAME 4

()PdfReport.ForGroups(...).Build

الـBuilder يرتب الأعمدة ويتحقق من الإعدادات وينتج GroupedPdfReportDefinition.

↓

FRAME 5

IPdfReportService → IPdfReportRenderer

تجهيز summary عند الحاجة، ثم تسليم التعريف إلى عقد الرسم.

↓

FRAME 6

QuestPdfReportRenderer

يرسم المجموعات والجداول ويحوّل المستند إلى PDF bytes.

↓

FRAME 7

HTTP application/pdf

الـendpoint يرسل الملف للمتصفح باسم grouped-sales-report.pdf.

٤ / أين QuestPDF؟

هذا هو الـAdapter: بيانات التقرير على اليسار مفاهيم يملكها التطبيق؛ أوامر الرسم على اليمين تخص QuestPDF.

`PageSizes.A4.Landscape()`	→	`report.Orientation`

الاتجاه المحايد يصبح إعداد صفحة QuestPDF.

`ConstantColumn / RelativeColumn`	→	`column.Width`

قيمة العرض المحايدة تحدد تعريف العمود.

`header.Cell().Text(...)`	→	`column.Header`

عنوان العمود يكتب داخل خلية الرأس.

`table.Cell().Text(...)`	→	`column.GetValue(row)`

قيمة الصف تتحول إلى محتوى خلية.

`cell.Hyperlink(url)`	→	`column.GetLink(row)`

الرابط اختياري؛ الخلية تعرض نصًا عاديًا بدونه.

`QuestPdfReportRenderer`	→	`IPdfReportRenderer`

التنفيذ الحالي واحد من التطبيقات الممكنة للعقد.

علامات QuestPDF في الملف

`using QuestPDF.*` | `Document.Create` | `IContainer` | `TableDescriptor` | `GeneratePdf()`

لا نمرر Document أو IContainer إلى Application. عند استبدال المكتبة، يتغير هذا الـAdapter وتسجيل DI، وليس وصف التقرير.

٥ / أسئلتك وإجاباتها

هل QuestPDF مربوط بمنطق التقرير؟

لا. Application يتعامل مع IPdfReportRenderer وتعريفاتنا. التنفيذ الذي يستخدم QuestPDF داخل Infrastructure.

هل كل تقرير جديد يحتاج Renderer جديدًا؟

لا. التقرير الجدولي الجديد يعيد استخدام renderer الجدولي. Renderer مستقل يلزم عندما يكون تخطيط المستند مختلفًا فعلًا.

هل Expressions تصمم شكل المستند؟

لا. هي تختار خاصية أو predicate بطريقة typed؛ الـBuilder يحفظها، والـrenderer يطبقها بصريًا.

هل نحتاج Strategy أو switch لأربعين تقريرًا؟

ليس لمجرد العدد. الـendpoint يستدعي خدمة التقرير، وDI يختار تنفيذ عقد الرسم. نقسم التقارير حسب عائلات layout.

هل هذا الـPoC يثبت كل تقارير الإنتاج؟

لا. يثبت حدود التقارير الجدولية flat/grouped. القوالب الخاصة مثل فاتورة مصممة بالكامل تحتاج مسارًا منفصلًا.

هل ملف PDF فيه حركة فعلية؟

لا. الحركة ممثلة بإطارات متتابعة مرقمة؛ PDF العادي لا يشغل animation بشكل موثوق.

٦ / كيف يكبر التصميم؟

عند وجود تقارير كثيرة، لا نبني Builder واحدًا لكل شكل مستند. نجمع التقارير المتشابهة في عائلة ونعيد استخدام renderer العائلة.

FAMILY A

Tabular reports

General Ledger وTrial Balance: تعريفات أعمدة مختلفة، ومركّب جدول مشترك.

FAMILY B

Invoice documents

InvoiceData + قالب فاتورة متخصص. لا نمرره إلى Builder الجدول.

FAMILY C

Custom layouts

تقرير بتصميم خاص يأخذ Composer مستقلًا، ويعيد استخدام header/footer عند وجودهما.

عند استبدال QuestPDF

```
تطبيق IPdfReportRenderer تحديث + آخر DI في Infrastructure
```

ما الذي لا يتغير؟ endpoints وخدمات Application وتعريفات التقرير، طالما renderer البديل يدعم الميزات التي يستخدمها التقرير.

حدود PoC الحالي: لا يرحّل 59 موضعًا، ولا ينفذ كل قوالب الفواتير، ولا يحتوي مكتبة PDF ثانية. يثبت الفكرة على flat/grouped فقط.

٧ / خريطة مذاكرة الكود

اقرأ الكود بهذا الترتيب. في كل خطوة اسأل: ما الذي يدخل؟ ما الذي يخرج؟ وأي طبقة تعرف تفاصيل QuestPDF؟

١	**Program.cs** DI registrations واتجاه ربط الخدمات.
٢	**ReportEndpoints.cs** .HTTP route → Application service → File response
٣	**SampleSalesReportService.cs** بيانات المثال واستخدام Fluent API.
٤	**PdfReport.cs + GroupedPdfReport.cs** Build، column order، Expressions، وتعريف التقرير النهائي.
٥	**IPdfReportRenderer + PdfReportService** العقد المحايد وتجهيز summary قبل الرسم.
٦	**QuestPdfReportRenderer.cs** تحويل التعريف المحايد إلى صفحة وجدول وPDF.
٧	**Application.Tests** التأكد من التعريفات والخدمات باستخدام fake renderer.

تمرين ورقة وقلم

```
تتبّع Amount: selector → GetValue → FormatValue → table cell → PDF
```